UCS503 – Software Engineering

UberLite

Design and Implementation of a Full-Stack Ride-Sharing Management Prototype

Submitted by:

1024030740 Ayush
1024030749 Anshika
1024030325 Gurleen Kaur

Group: 3C53, BE, Computer Engineering

Submitted to:

Dr. Paramveer Kaur

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **3**
1.1 Background and Context . . . 3
1.1.1 Problem Statement . . . 3
1.2 Project Objectives . . . 3

2 Methodology and Design **5**
2.1 System Architecture . . . 5
2.1.1 High-Level Design . . . 5
2.1.2 Technical Stack . . . 6
2.2 Implementation Details . . . 7
2.2.1 Module 1: Core Functionality . . . 7
2.2.2 Module 2: Secondary Features . . . 9

3 Results and Evaluation **11**
3.1 Testing Methodology . . . 11
3.2 Performance Metrics . . . 11
3.3 Technical Development Progress . . . 12
3.4 Analysis . . . 12

4 Conclusion **14**
4.1 Summary of Work . . . 14
4.2 Key Findings . . . 14
4.3 Future Work and Recommendations . . . 14
4.4 Final Remarks . . . 15

Bibliography **16**

Chapter 1

Introduction

1.1 Background and Context

Ride-sharing applications provide a digital platform through which passengers can request transportation and drivers can accept and complete trips. Such applications require coordination between passengers, drivers, location information, ride status, and fare calculation.

A ride-sharing system must handle several connected operations, including user authentication, driver availability, assignment of drivers to ride requests, tracking of ride progress, and maintaining ride records. These operations must be coordinated to provide a consistent experience for both passengers and drivers.

UberLite is a full-stack ride-sharing management prototype developed to demonstrate these core operations. The system provides separate passenger and driver interfaces supported by a backend API and a relational database. It incorporates location-based driver matching, distance-based fare estimation, controlled ride-state transitions, and post-ride feedback.

The current implementation focuses on establishing a complete and testable ride-booking workflow. More advanced capabilities, including WebSocket-based updates and optimized spatial searching, are identified as future enhancements.

1.1.1 Problem Statement

A ride-sharing application must coordinate ride requests with available drivers while maintaining consistent ride status and driver availability. A poorly coordinated system may assign conflicting requests to the same driver, permit invalid ride-state transitions, or calculate fares inconsistently.

The problem addressed by UberLite is the design and implementation of a modular ride-sharing prototype that connects passenger requests, driver assignment, ride execution, fare calculation, and feedback within a unified full-stack application.

The system must enforce role-based access, validate incoming requests, preserve consistent ride-state transitions, and store relevant information in a relational database. It should also provide a foundation for future improvements in real-time communication, matching efficiency, and scalability.

1.2 Project Objectives

The project has the following objectives:

1. **Authentication and authorization:** Implement registration and login for passengers and drivers, with JWT-based authentication and role-based access control.

2. **Location-based driver matching:** Develop a location utility using the Haversine formula and assign the nearest eligible available driver to a ride request.

3. **Ride lifecycle management:** Implement the ride workflow from request through acceptance and initiation to completion, while rejecting invalid state transitions.

4. **Fare calculation:** Implement fare estimation based on a base fare and the calculated travel distance, with final fare calculation upon ride completion.

5. **Full-stack integration:** Connect the passenger and driver dashboards to backend APIs and persist application data using PostgreSQL and Prisma ORM.

6. **Testing and verification:** Validate core functionality through unit and integration tests and verify that the application builds successfully.

The objectives are intended to be achieved incrementally, with the initial phase prioritizing a working end-to-end ride-booking workflow before advanced optimization and real-time features.

Chapter 2

Methodology and Design

2.1 System Architecture

UberLite follows a modular full-stack architecture. The frontend provides user interfaces for passengers and drivers, while the backend exposes APIs for authentication, driver management, ride operations, fare calculation, and feedback.

The backend communicates with a PostgreSQL database through Prisma ORM. Request validation and authentication middleware are used to validate incoming data and restrict access to role-specific operations.

2.1.1 High-Level Design

The system consists of the following major components:

1. **Frontend layer:** The React application provides registration and login, passenger and driver dashboards, ride-request controls, ride-status information, history, and feedback interfaces.

2. **API and middleware layer:** The Express backend handles HTTP requests, authentication, role-based authorization, request validation, and error responses.

3. **Service layer:** Modular services implement location calculations, driver matching, ride-state transitions, fare calculation, and other business operations.

4. **Data layer:** PostgreSQL stores application records, while Prisma ORM provides database access, schema management, and transaction support.

5. **Testing and deployment layer:** Automated tests verify application behaviour, and Docker and deployment configurations support preparation for demonstration and deployment.

The principal request flow is:

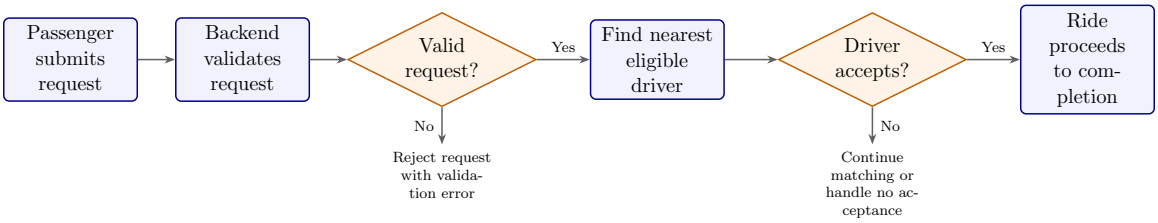


Figure 2.1: Principal request flow in UberLite

Responses travel back through the backend API to the relevant frontend interface. The current dashboard implementation uses HTTP polling for ride-status updates rather than a WebSocket connection.

2.1.2 Technical Stack

The technologies used in the reported implementation are summarized in Table 2.1.

Component	**Technology**	**Purpose**
Frontend	React 19, TypeScript, Vite	Provides passenger and driver interfaces.
Backend	Node.js, Express	Handles API requests and business logic.
Database	PostgreSQL	Stores persistent application and ride data.
ORM	Prisma ORM	Provides structured database access.
Request validation	Zod	Validates incoming request data.
Authentication	JSON Web Tokens (JWT)	Supports token-based authentication.
Location calculation	Haversine formula	Estimates distance between coordinates.
Testing	Unit and integration tests	Checks core application behaviour.
Infrastructure	Docker, Docker Compose	Supports containerization and deployment preparation.

Table 2.1: Technology stack and component responsibilities

2.2 Implementation Details

The implementation was organized into modular components so that the core ride-booking workflow could be developed and tested incrementally.

2.2.1 Module 1: Core Functionality

Authentication and Authorization

The authentication module supports registration and login for passengers and drivers. Incoming requests are validated, and authenticated sessions use JSON Web Tokens. Role-based middleware restricts protected operations to the appropriate user role.

Frontend route guards provide corresponding protection for passenger and driver application areas. Figure 2.2 shows the sign-in interface, while Figure 2.3 shows the role-specific registration forms for passengers and drivers.

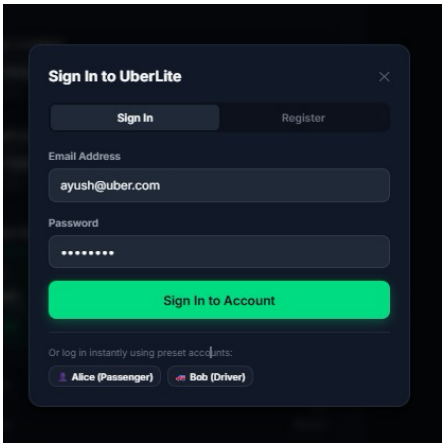
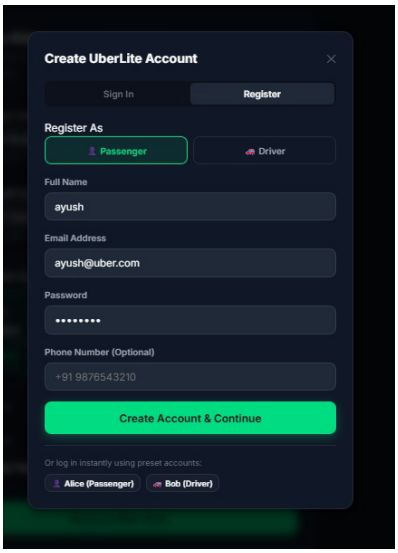
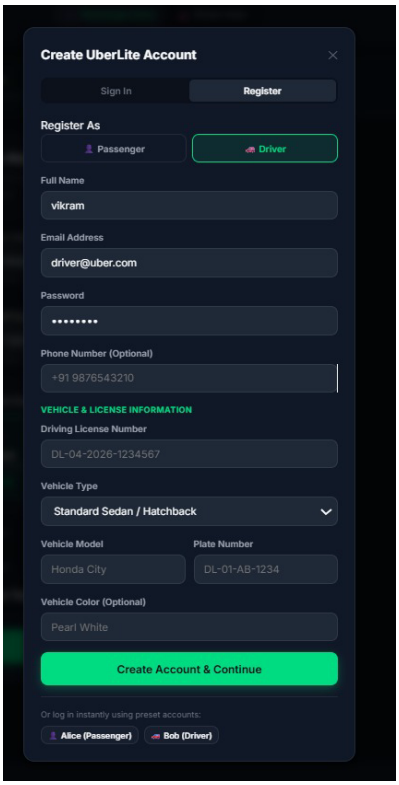


Figure 2.2: UberLite sign-in interface with passenger and driver demo-account options.



(a) Passenger registration



(b) Driver registration

Figure 2.3: Role-specific registration forms in UberLite. The driver form additionally collects vehicle and driving-licence information.

Location Calculation and Driver Matching

The location utility calculates the distance between two geographic coordinates using the Haversine formula [Mov26]. For coordinates represented by latitude ϕ and longitude λ, the calculation uses:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right) \tag{2.1}$$

$$d = 2R\arcsin\left(\sqrt{a}\right) \tag{2.2}$$

where R is the assumed Earth radius, the angles are expressed in radians, and d is the calculated great-circle distance.

The matching service considers available drivers, calculates their distance from the passenger's pickup location, and selects the nearest eligible driver. The initial

matching approach uses a linear search. Transaction-safe assignment logic is used to reduce the risk of assigning the same driver to conflicting ride requests.

Ride Lifecycle Management

A ride progresses through the following states:

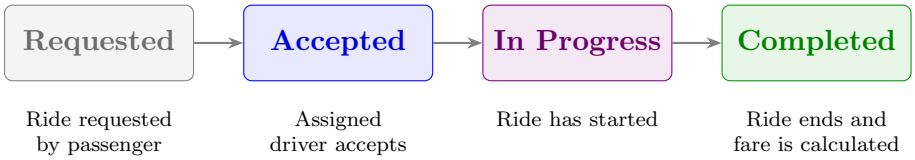


Figure 2.4: Ride lifecycle and valid state transitions in UberLite.

The backend validates state transitions and rejects invalid operations, such as completing a ride that has not started. Driver availability is updated as the ride starts and finishes, helping prevent conflicting assignments.

Fare Calculation

The fare module estimates the cost of a ride using a base fare and a distance-based rate. The fare model is:

$$F = B + (D \times r) \tag{2.3}$$

where F represents the fare, B is the base fare, D is the distance used for fare calculation, and r is the rate per unit distance.

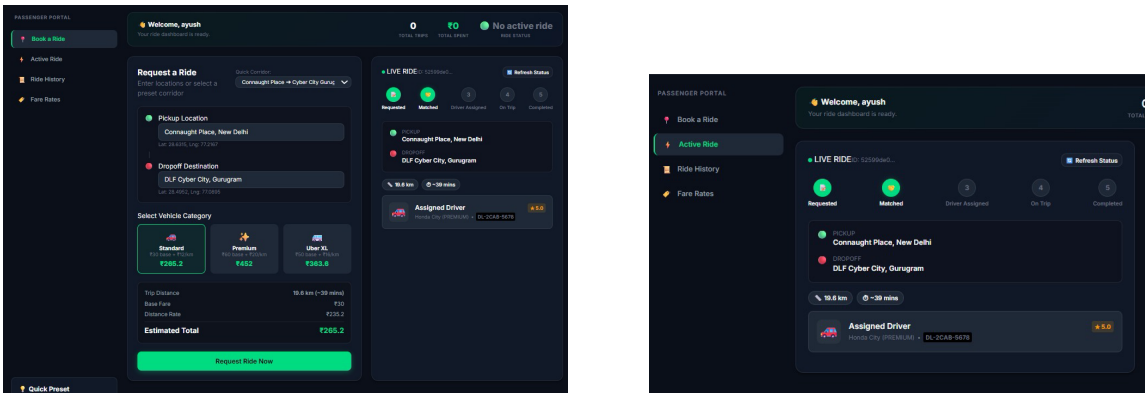
The final fare is calculated when the ride is completed and persisted with the ride record. The configured fare parameters are maintained by the application.

2.2.2 Module 2: Secondary Features

Passenger and Driver Dashboards

The passenger dashboard supports ride requests, estimated fare information, ride-status updates, driver information, and post-ride feedback. The driver dashboard supports availability controls, incoming ride requests, and actions for accepting and progressing assigned rides.

The dashboards communicate with the backend through HTTP APIs. Polling is used to refresh relevant ride information. Figure 2.5 illustrates the ride-request view, including pickup and drop-off locations and fare estimation, alongside the active-ride view with progress status and assigned-driver information.



(a) Ride request and fare estimate

(b) Active ride and assigned driver

Figure 2.5: Passenger dashboard views in UberLite.

Ride History and Feedback

The application includes ride-history APIs and corresponding history interfaces for passengers and drivers. Relevant information includes ride details, status, fare, and completion information.

After a ride has been completed, the passenger can submit a rating and optional feedback associated with the ride and its driver.

Validation and Error Handling

Zod is used for request validation, while centralized backend error handling provides consistent responses for invalid input, unauthorized operations, missing resources, and invalid workflow transitions.

Database relationships, constraints, and transaction handling support data consistency across related operations.

Integration and Deployment Preparation

The frontend and backend have been integrated to support the main ride-booking workflow. The project also contains Dockerfiles, Docker Compose orchestration, and configurations for deployment platforms.

These configurations provide deployment preparation; they should not be interpreted as evidence of a publicly deployed production service.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Testing was carried out to verify the correctness of the individual modules and their interaction across the full-stack application.

- **Unit testing:** Tests cover individual operations, including location calculations, fare calculation, matching logic, ride-state transitions, and validation rules.
- **Integration testing:** Tests cover connected workflows such as registration, login, role-based access, ride creation, driver matching, ride progression, fare persistence, history, and feedback.
- **Frontend testing:** Tests validate session storage, protected routes, form validation, and authentication-related UI states.
- **Build verification:** Production TypeScript compilation and application builds are used to identify compilation and build errors.
- **Performance evaluation:** Backend instrumentation records relevant timing and workflow information. A separate load-testing study remains part of the future scope unless supported by additional measured results.

3.2 Performance Metrics

The current evaluation emphasizes automated test results and build verification. Table 3.1 summarizes the results reported for the current prototype.

Metric	**Reported result**	**Status**
Backend automated tests	307 passing	Reported pass
Frontend automated tests	34 passing	Reported pass
Total passing tests	341	Reported pass
Production TypeScript build	Zero compilation errors	Reported pass

Table 3.1: Reported testing and build results

The reported test results indicate that the prototype has undergone automated verification across backend functionality and frontend behaviour. The combined test count is 341, based on the reported backend and frontend suite totals.

These results establish evidence of functional verification, but do not by themselves establish production-scale throughput, average response time, or real-world matching accuracy. Those metrics require dedicated measurement under defined test conditions.

3.3 Technical Development Progress

The technical development is organized into the core application workflow followed by verification and planned engineering improvements. The first stages establish the database and backend, authentication, ride management, and frontend integration. Testing and validation are used to verify the implemented functionality. Real-time communication and performance optimization remain planned improvements.

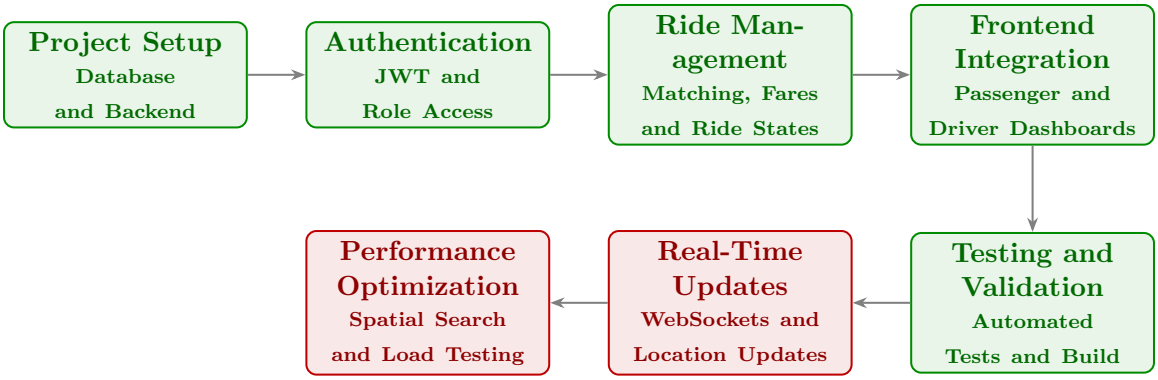


Figure 3.1: Technical development progress and planned improvements in UberLite

3.4 Analysis

The current implementation addresses the principal objectives of the project by providing an integrated ride-booking workflow. Authentication and authorization separate passenger and driver operations, while backend validation restricts invalid requests and ride-state transitions.

The location-based matching service provides a functional baseline for assigning available drivers. However, its linear search approach may become less efficient as the number of available drivers increases. Spatial indexing or bounding-box-based filtering can be investigated in a subsequent phase.

The current polling mechanism supports ride-status synchronization but introduces repeated HTTP requests. WebSocket-based communication could reduce unnecessary polling and provide more immediate updates.

The implementation also includes transaction-safe driver assignment to improve consistency during concurrent requests. Further concurrency and load testing is needed to quantify its behaviour under heavier workloads.

Overall, the prototype provides a foundation for evaluating the core ride-booking workflow. Additional performance measurements, broader user evaluation, and deployment testing would strengthen the assessment beyond functional correctness.

Chapter 4

Conclusion

4.1 Summary of Work

UberLite has progressed from initial project setup to an integrated full-stack ride-sharing prototype. The reported implementation includes a React and TypeScript frontend, an Express backend, and a PostgreSQL database managed through Prisma ORM.

The completed core workflow covers passenger and driver authentication, role-based authorization, location-based driver matching, ride-state transitions, fare calculation, dashboards, ride history, and feedback.

Automated backend and frontend tests have been reported as passing, and production build verification has been reported as successful. These results provide an initial basis for evaluating the correctness of the implemented functionality.

4.2 Key Findings

The current implementation supports the following findings:

- A modular full-stack architecture can connect authentication, driver matching, ride management, and fare calculation into one workflow.
- Backend validation, role-based authorization, and controlled ride-state transitions help enforce application rules and restrict invalid operations.
- Automated tests provide evidence of functional correctness, while further performance testing is required to evaluate scalability and response times under heavier workloads.

4.3 Future Work and Recommendations

The next phase will focus on improving the efficiency, responsiveness, and scalability of the prototype.

1. **Driver search optimization:** Replace or optimize the initial linear driver search using spatial indexing or bounding-box-based queries.
2. **Real-time communication:** Introduce WebSockets for passenger and driver ride-status synchronization while retaining the REST API as the source of ride state.

3. **Driver location updates:** Implement periodic location updates to improve the freshness of driver coordinates used for matching and future ride tracking.
4. **Concurrency and history improvements:** Extend concurrent ride-request testing and add practical filtering to ride-history interfaces.
5. **Frontend improvements:** Improve responsiveness, usability, loading states, and error presentation across different screen sizes.
6. **Performance and scalability evaluation:** Measure API response times, driver-assignment latency, and system behaviour under increasing numbers of drivers and concurrent ride requests.
7. **Deployment and documentation:** Verify the deployment workflow and prepare supporting architecture diagrams, API documentation, and demonstration instructions.

4.4 Final Remarks

The current phase establishes the functional foundation of UberLite as a full-stack ride-sharing management prototype. It demonstrates the integration of frontend interfaces, backend services, database persistence, driver matching, and ride lifecycle management.

The subsequent development phase will build on this foundation by improving real-time communication, search efficiency, interface responsiveness, and performance evaluation. These enhancements will help determine how effectively the prototype can be extended beyond its current core functionality.

Bibliography

[Mov26] Movable Type Scripts. *Calculate Distance, Bearing and More Between Latitude/Longitude Points*. 2026. URL: `https://www.movable-type.co.uk/scripts/latlong.html` (visited on 10/09/2026) (cit. on p. 8).